

Laurentian University
Université Laurentienne

SCHOOL OF ENGINEERING AND COMPUTER SCIENCE

Modern Processor Case Study: How AMD's Zen 5 Architecture Delivers Performance in the Ryzen 9000 Series

COSC 3406 – Computer Organization

Faculty Advisor:
Prof. Williams Nwadiugwu

Fall, 2026.

Submitted By Team 1

October 19, 2026

Chapter 1 – Project Synopsis, and Literature Review.

1.1 Project Synopsis

This case study looks at one modern desktop processor, the AMD Ryzen 9 9950X, and asks a simple question: if clock speeds are no longer going up, where does the extra performance of a new chip come from? The 9950X was released in August 2024 and is built on AMD's Zen 5 core. We picked it because it brings together several topics from our course in one product. It has a wide out-of-order core, it supports vector (SIMD) instructions that

work on 512 bits at a time, and it is not made from one piece of silicon but from a few smaller chiplets joined together.

Our main finding is that almost all of the 9950X's gain over the older Ryzen 9 7950X comes from how the chip is organized, not from its clock or core count, since both chips have 16 cores and the same 5.7 GHz top clock. In our worked example, the wider Zen 5 core runs the same program about 14% faster. Our simple cost model shows that building the chip from small chiplets gives about 2.4 times more working processors per wafer than one large die would. We did not have a 9950X to test, so all of our numbers come from AMD's Hot Chips 2024 presentation (Cohen & Subramony, 2024), from the two papers we reviewed, or from examples we worked out ourselves. Whenever a value is our own assumption, we say so.

1.2 Why Faster Clocks Stopped Being Enough

Chapter 2 of our textbook explains the background to this project. For many years, the number of transistors and the clock speed went up together. Around 2004 the clock speed stopped rising, even though the number of transistors kept growing. The main reason is heat. A chip uses more power as its clock gets faster, and after a certain point it simply cannot be cooled. Smaller and thinner wires also add RC delay, which slows signals down even when the transistors themselves get faster.

The 9950X shows how stuck the clock has become. Its top boost speed is 5.7 GHz, exactly the same as the Ryzen 9 7950X that came out two years earlier. So if AMD wanted the new chip to be faster, the gain had to come from somewhere else: doing more work in each clock cycle, using more cores, and keeping those cores supplied with data.

1.3 Literature Review: Chiplets at AMD

The first paper we reviewed is by Naffziger et al. (2021), a group of AMD engineers who worked on the company's chiplet design. They explain that every new manufacturing process costs more per square millimetre than the one before, and that a large chip is more likely to have a defect that makes it useless. They also point out that not every part of a chip benefits from a newer process. The cores and caches shrink and get faster, but the circuits that connect to memory and I/O hardly shrink at all. AMD's first EPYC server chips in 2017 used four identical dies in one package. In 2019 AMD went further and split the design into small core chiplets on the newest process plus one I/O die on an older, cheaper process. The authors are honest about the downsides as well. The links between chiplets take extra area and power, and data that has to move from one die to another takes longer to arrive.

1.4 Literature Review: Amdahl's Law in the Multicore Era

The second paper is by Hill and Marty (2008). They start from Amdahl's Law, which says that the part of a program that cannot run in parallel limits how much extra cores can help. To this they add a simple cost model. A chip has a fixed budget of resources, and the designer can spend it on many small cores or on fewer, bigger and faster ones. They compare three kinds of chips: chips where all cores are the same, chips with one big core and many small ones, and chips that can combine their resources into one strong core when a program needs it. Their results show that even for programs that are mostly parallel, a few strong cores often do better than a large number of weak ones, and the mixed and combining designs do best of all. Their main message is that making each core faster still matters.

1.5 How the Two Papers Fit Together

Together, the two papers explain the design of the 9950X quite well. Hill and Marty (2008) explain why a modern chip needs both strong cores and many of them, which is what AMD did by making each Zen 5 core wider while still offering 16 of them. Naffziger et al. (2021) explain how AMD can afford to put that many cores into a desktop chip in the first place. Neither paper covers Zen 5 itself, since both were written before it existed, so we used AMD's Hot Chips presentation to connect their ideas to the real chip.

Chapter 2 – System/architecture diagram.

2.1 The Case-Study Processor and Its Package

Figure 1 shows how the Ryzen 9 9950X is put together. The package holds three dies. Two of them are core complex dies (CCDs), and each CCD has eight Zen 5 cores plus a 32 MB L3 cache that those eight cores share. Both CCDs are made on TSMC's N4P process. The third die is the I/O die, made on TSMC's older N6 process. It contains the DDR5 memory controller, the PCIe 5.0 lanes, USB and a small graphics unit. AMD reused this same I/O die from the Ryzen 7000 series, which is a good real example of the kind of reuse Naffziger et al. (2021) describe.

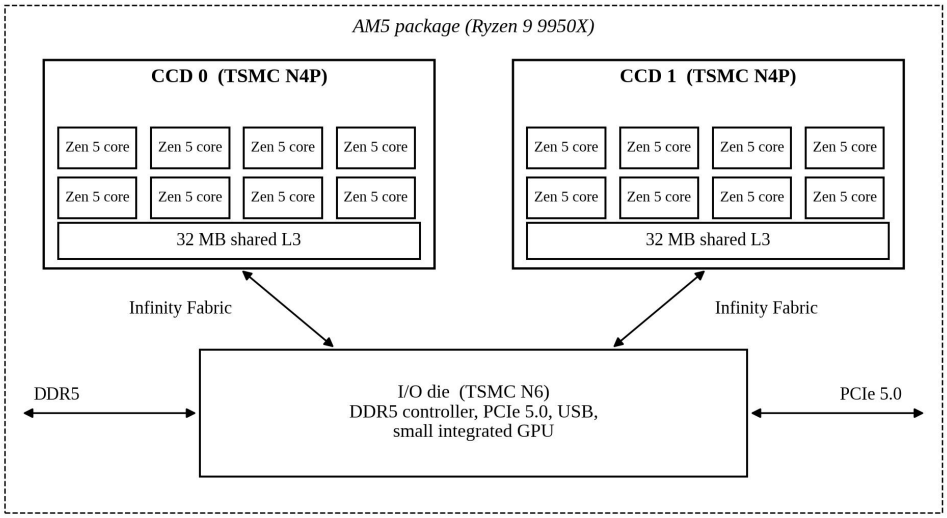


Figure 1. Layout of the Ryzen 9 9950X package (drawn by us).

The dies are connected by AMD's Infinity Fabric. This is a point-to-point connection, which is the same idea as the QuickPath Interconnect example in Chapter 3 of our textbook. Instead of all the parts sharing one bus and waiting their turn, each die has its own link and sends data in packets. The downside is that when a core needs something from main memory, the request has to travel from its CCD to the I/O die and back again.

2.2 Inside One Zen 5 Core

Figure 2 shows what happens inside a single core. We drew it from the block diagrams in Cohen and Subramony (2024) and kept only the parts we discuss in this report.

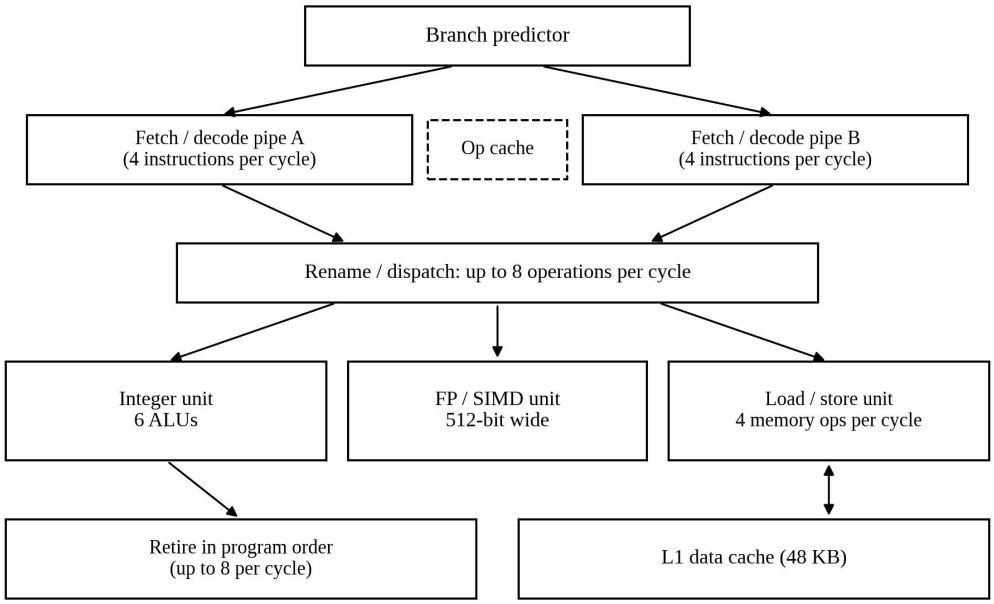


Figure 2. Main stages of one Zen 5 core, based on Cohen and Subramony (2024) (drawn by us).

The branch predictor comes first. It guesses which way the program will go next, so the core can keep fetching instructions without stopping to wait. The instructions are then fetched and decoded into simpler internal steps called operations. Zen 5 has two decode pipes working side by side, and each one can handle four instructions per cycle. There is also an op cache, which keeps instructions that were already decoded so that loops do not have to be decoded again every time.

Next, up to eight operations per cycle are sent to the back end of the core, where the actual work happens. The back end has an integer unit with six ALUs (the parts that do arithmetic and logic), a floating-point and SIMD unit that is 512 bits wide, and a load/store unit that can do four memory reads or writes per cycle using a 48 KB L1 data cache. At the end, the results are retired, or made final, in the original program order, up to eight per cycle. Each core can also run two threads at the same time using simultaneous multithreading (SMT), so the 16-core chip appears to the operating system as 32 processors.

2.3 Full Technologies/Tools Stack and System Requirements

Table 1 lists the hardware we studied, the sources we relied on and the tools we used to put this report together.

Table 1. Technologies and tools involved in this case study.

Layer	Technology / tool	Role in our study
Processor	AMD Ryzen 9 9950X, Zen 5, 16 cores / 32 threads	The real-world system we analyze
Packaging	2 CCDs (TSMC N4P) and 1 I/O die (TSMC N6)	Chiplet design, linked to Naffziger et al. (2021)
Interconnect	Infinity Fabric	Connects the CCDs to the I/O die
Instruction set	x86-64 with AVX-512 (512-bit datapath)	Used in our vector example
Memory and I/O	Dual-channel DDR5-5600, PCIe 5.0	Used in our bandwidth calculation
Performance models	Processor time equation, Amdahl's Law, Hill and Marty model, Little's Law	Our quantitative analysis
Manufacturing model	Simple defect-based yield formula	Our chiplet cost estimate
Our own tools	Python 3 with Matplotlib, Microsoft Word	Calculations, figures and this report

To build a computer around the 9950X, you need an AM5 motherboard, DDR5 memory and a separate CPU cooler, because AMD does not include one with this chip. The 9950X is rated at 170 W, so the cooler and the power supply need to be able to handle that. Software matters as well. A program has to be compiled with AVX-512 support to use the wide vector unit, and the operating system needs to understand the two-CCD layout so it can place threads sensibly. For our own calculations, a normal laptop with Python installed was all we needed.

2.4 Technical Comparison Table

Table 2 compares the 9950X with the chip it replaced, the Ryzen 9 7950X.

Table 2. Ryzen 9 7950X (Zen 4) and Ryzen 9 9950X (Zen 5), from Cohen and Subramony (2024) and AMD's product specifications.

Feature	Ryzen 9 7950X (Zen 4)	Ryzen 9 9950X (Zen 5)
Cores / threads	16 / 32	16 / 32
Maximum boost clock	5.7 GHz	5.7 GHz
Rated power (TDP)	170 W	170 W
Core chiplet process	TSMC 5 nm	TSMC N4P (4 nm class)
Decode pipes per core	1	2 (4 instructions each)
Dispatch width	6 operations per cycle	8 operations per cycle
Integer ALUs per core	4	6
L1 branch target buffer	1.5K entries	16K entries
AVX-512 execution	Two 256-bit halves	Full 512 bits in one pass

The first three rows are identical for both chips: the same number of cores, the same clock and the same power rating. This means that whatever AMD gained with Zen 5 has to come from the rows below them, and all of those are about how the core itself is built.

Chapter 3 – Critical evaluation and Real-world case study.

3.1 Case-Study System Description

In this section we explain how the main parts of the processor do their job, using small examples to make each one easier to follow.

Running a loop

Imagine a short loop that goes through a list of prices, multiplies each price by a tax rate and adds the result to a total. A simple in-order processor would finish one pass of the loop before it starts the next one. On Zen 5, the branch predictor quickly learns that the loop usually repeats, so the core starts fetching the next few passes early. The load unit begins reading the prices needed later, the multiplications from several passes run on different units at the same time, and only the additions to the total have to wait for one another. Every instruction still goes through the fetch and execute cycle from Chapter 3 of our textbook, but many of these cycles now overlap.

Wrong guesses

All of this only works because the core is allowed to guess. When the loop finally ends, the predictor will usually have guessed that it keeps going, so the core has already started work that should never happen. That work is thrown away, and fetching starts again from the correct place. Because a wider core does more work in each cycle, it also throws away more on every wrong guess. We think this is a big reason why AMD made the branch target buffer so much larger in Zen 5 (see Table 2).

Vector instructions

AVX-512 lets a single instruction work on a whole group of numbers at once. A 512-bit register holds sixteen 32-bit numbers, so one instruction can multiply sixteen prices by the tax rate together. Zen 4 could run the same instruction, but it had to split it into two 256-bit halves. Zen 5 handles it in one pass.

Two threads on one core

With SMT, two threads share the same execution units inside one core. If one thread is stuck waiting for data from memory, the other one can use the ALUs in the meantime. It does

not double the speed, though, because the two threads are competing for the same caches and units. A program that already keeps a core fully busy gains very little from a second thread.

Moving between chiplets

Each CCD has its own L3 cache, and the two CCDs do not share theirs. If two threads that work on the same data end up on different CCDs, every exchange between them has to go from one CCD, through the I/O die, to the other. Windows and Linux both try to keep related threads on the same CCD for this reason. This is one of the costs of chiplets that Naffziger et al. (2021) warn about.

3.2 Quantitative Analysis and Performance Metrics

System Results/Output Constraints/Parameters:

Values taken from our sources: 16 cores, a 5.7 GHz top clock, an average of about 16% more instructions per clock (IPC) than Zen 4 (Cohen & Subramony, 2024), 512-bit vectors, and dual-channel DDR5-5600 memory.

Values we assumed to keep the examples simple: a 5.0 GHz working clock, an IPC of 2.0 for Zen 4 in our example program, a memory delay of about 80 ns, a defect density of 0.1 defects per square centimetre, and rough die sizes of 71 mm² for one CCD and 264 mm² for an imagined 16-core chip built as one single die. Real programs behave very differently from each other, so these numbers are only meant to show how the ideas work, not to predict exact results.

i) Operational/Transitional Efficiency

Processor time

Our textbook gives this formula for how long a program takes to run, where Ic is the number of instructions, CPI is the average number of clock cycles per instruction, and τ is the length of one clock cycle:

$$T = Ic \times CPI \times \tau$$

Suppose a program has 100 billion instructions and runs on Zen 4 with an IPC of 2.0, which means a CPI of 0.5. At 5.0 GHz, one cycle lasts 0.2 ns. This gives $T = 10^{11} \times 0.5 \times 0.2$ ns = 10 seconds. With Zen 5's 16% higher IPC, the IPC becomes 2.32 and the CPI drops to about 0.431, so $T = 10^{11} \times 0.431 \times 0.2$ ns ≈ 8.6 seconds. The same program finishes about 14% sooner, even though the clock and the number of instructions did not change. The whole gain comes from the CPI part of the formula, which is exactly the part a wider core is designed to improve.

Vector work

Now suppose the program adds two lists of one million 32-bit numbers. With normal one-at-a-time instructions, this takes one million add instructions. With AVX-512, each instruction handles 16 numbers, so it takes only 1,000,000 ÷ 16 = 62,500 instructions. On Zen 4, each of those instructions goes through the hardware twice, which adds up to 125,000 passes. On Zen 5 it is 62,500 passes, so half the hardware work for exactly the same code.

Memory bandwidth and Little's Law

Dual-channel DDR5-5600 can move 8 bytes per transfer on each of its two channels, so the most data it can deliver is:

5,600 million transfers/s × 8 bytes × 2 channels = 89.6 GB/s

When all 16 cores are busy, that works out to only about 5.6 GB/s for each core. Little's Law from Chapter 2 of our textbook tells us how many requests need to be in progress at once to actually use all of this bandwidth. It says that the number of items in a system equals the rate at which they arrive multiplied by how long each one stays. With an 80 ns memory delay, the amount of data on its way at any moment is 89.6 GB/s × 80 ns ≈ 7,170 bytes. Since each request brings 64 bytes, that means about 112 memory requests have to be in progress at the same time. A core that ran one instruction at a time could never keep that many requests going. For us, this was one of the clearest reasons why out-of-order execution matters: the core keeps working past a slow load, so it can send out many more loads before the first one even comes back.

ii) Performance, Scalability, Cost, and Energy Implications

Performance

AMD's figure of 16% more IPC is an average over many different programs (Cohen & Subramony, 2024). In the reviews we read, the actual results varied a lot. Programs that use AVX-512 improved by much more than 16%, while many games improved by only a few percent, because games depend more on memory delays than on how wide the core is. This is a good example of why our textbook warns against trusting a single benchmark number.

Scalability

Figure 3 plots Amdahl's Law, speedup = 1 / ((1 − f) + f / N), where f is the fraction of the program that can run in parallel and N is the number of cores.

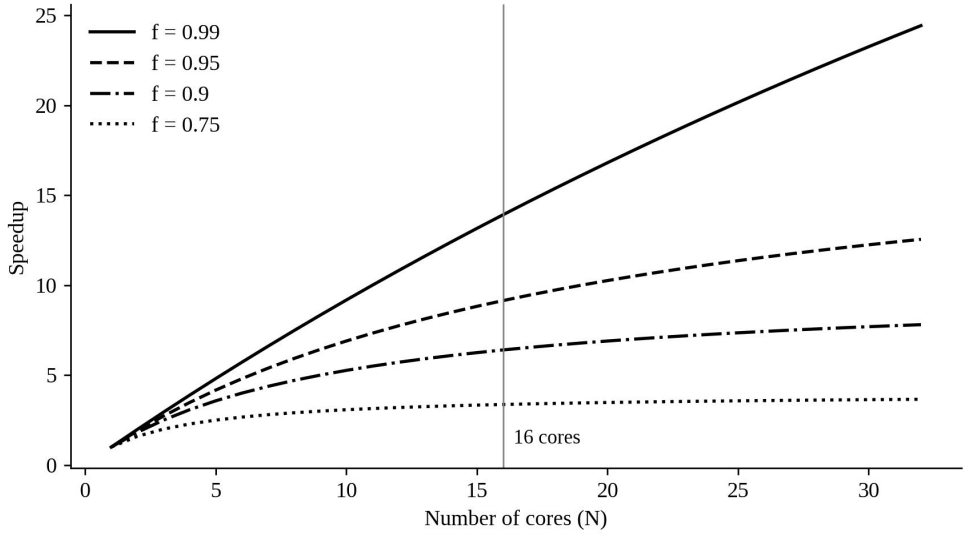


Figure 3. Amdahl's Law speedup for different parallel fractions f (calculated by us).

With 16 cores, a program that is 90% parallel runs only about 6.4 times faster than it would on one core, and even a 99% parallel program reaches only about 13.9 times. We then compared two ways AMD could make the chip faster, following the idea in Hill and Marty (2008). Going from 8 to 16 cores raises the speedup of a 90% parallel program from about 4.7 to 6.4, which is a 36% gain. For a program that is only 50% parallel, the speedup goes from 1.78 to 1.88, a gain of just 6%. A core that is 16% faster, on the other hand, helps both programs by the full 16%, because it also speeds up the part that cannot run in parallel. This matches what Hill and Marty argue, and it explains why AMD made each core wider instead of only adding more cores. AMD's Ryzen AI 300 laptop chips go one step closer to the mixed design in their paper, since they put full Zen 5 cores and smaller Zen 5c cores on the same chip.

Cost

To see why chiplets save money, we used a simple yield formula. The chance that a die has no defects is Y = e^(−D × A), where D is the number of defects per square centimetre and A is the area of the die. With our assumed D = 0.1, a 71 mm² CCD comes out with no defects about 93% of the time. An imagined 264 mm² single-die 16-core chip on the same process manages only about 77%. A standard 300 mm wafer fits about 916 CCDs, so roughly 854 good ones, which is enough for 427 sixteen-core processors. The same wafer fits only about 227 of the large dies, and only about 174 of those would work. Figure 4 shows the difference.

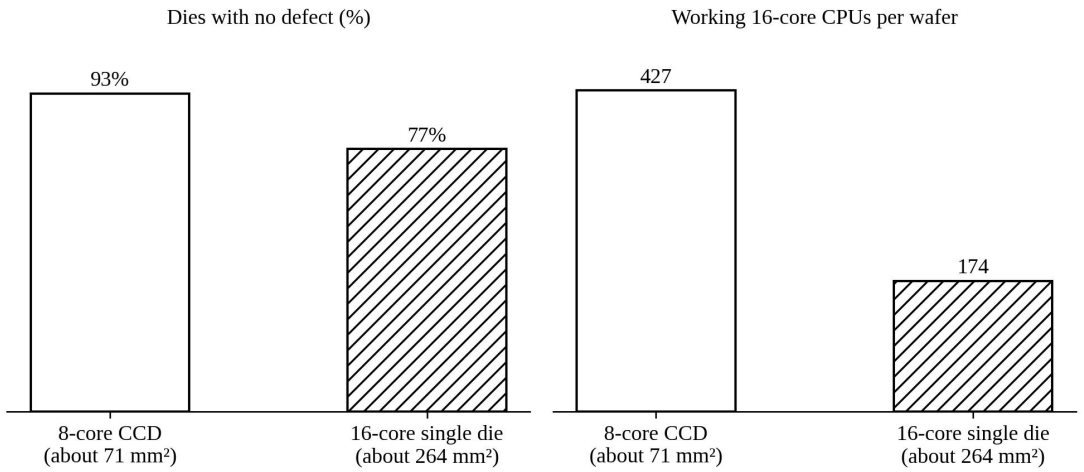


Figure 4. Estimated yield and number of working chips per wafer (our calculation, using assumed values).

By our estimate, chiplets give about 2.4 times as many working processors from each expensive 4 nm wafer. On top of that, the I/O die is made on a separate and cheaper wafer, and a CCD with one or two faulty cores can still be sold in a 6-core or 12-core model. Our model is very basic and leaves out the extra cost of packaging several dies together, but its result agrees with the main point of Naffziger et al. (2021).

Energy

The 9950X is rated at 170 W, the same as the 7950X. Doing about 16% more work at the same power rating means better performance per watt, even though the chip itself is not a low-power part. Chiplets do have an energy cost, though. Sending data between dies over Infinity Fabric uses more energy than keeping it inside one die, which Naffziger et al. (2021) also mention. For laptops, where battery life matters most, AMD uses the smaller Zen 5c core, which gives up some clock speed to save space and power.

Trade-offs

Each strength of this chip comes with a cost. Chiplets make the chip cheaper and make it easy to offer different core counts, but they add delay and use extra energy whenever data has to cross between dies. The wider core helps every program, but it needs more transistors and power, and it wastes more work on each wrong guess. Sixteen cores are a big help for parallel programs, but Amdahl's Law limits the gain, and all of the cores have to share just two memory channels. Finally, AVX-512 doubles vector speed, but only for programs that are built to use it.

Chapter 4 – Conclusion.

4.1 Conclusion

We started this project wanting to find out where a new processor's speed comes from when its clock is no longer getting faster. For the Ryzen 9 9950X, Table 2 gives a clear answer. It has the same 16 cores, the same 5.7 GHz clock and the same 170 W rating as the chip before it, so all of its improvement comes from how it is organized. The two decode pipes, the eight-wide dispatch, the six ALUs, the larger branch predictor and the full 512-bit vector unit all increase the amount of work done in each cycle. In our example, this cut a program's run time by about 14% with nothing else changed, and it halved the hardware passes needed for vector work.

The two papers helped us understand why AMD built the chip this way. Hill and Marty (2008), backed up by our own Amdahl's Law numbers, showed that a faster core helps every program, while extra cores only help the parts that run in parallel. Naffziger et al. (2021) showed how AMD can afford to put 16 cores into a desktop chip at all, and our yield estimate suggested that chiplets give about 2.4 times more working chips per wafer. Little's Law then showed us that the core has to be wide and out-of-order simply to keep enough memory requests going.

Our study also has some clear limits. We never tested a real 9950X. Our IPC, memory delay, defect density and die size values are either averages or our own assumptions, and real programs can behave very differently. Our yield model also ignores the cost of packaging. If we had more time and access to the hardware, we would run the same set of benchmarks on a 7950X and a 9950X at the same clock speed, with and without AVX-512, to measure how much each change adds on its own.

The biggest lesson we take from this project is that a specification sheet only tells part of the story. Two chips with the same core count, clock speed and power rating can perform quite differently, and the reason lies in how the core is built and how the pieces of the chip are connected.

4.2 References

Cohen, B., & Subramony, M. (2024, August). *Next generation "Zen 5" core* [Conference presentation]. Hot Chips 2024 (HC36), Stanford, CA, United States. https://www.hc2024.hotchips.org/assets/program/conference/day2/24_HC2024.AMD.Cohen.Subramony.final.pdf

Hill, M. D., & Marty, M. R. (2008). Amdahl's law in the multicore era. *Computer, 41*(7), 33–38. https://doi.org/10.1109/MC.2008.209 (Model code and online grapher: https://research.cs.wisc.edu/multifacet/amdahl/)

Naffziger, S., Beck, N., Burd, T., Lepak, K., Loh, G. H., Subramony, M., & White, S. (2021). Pioneering chiplet technology and design for the AMD EPYC™ and Ryzen™ processor families: Industrial product. In *Proceedings of the 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)* (pp. 57–70). IEEE. https://doi.org/10.1109/ISCA52012.2021.00014